What is Time Complexity?

Time Complexity measures how the running time of an algorithm grows as the input size **n** increases.

It helps estimate algorithm performance, especially for large inputs.

Common Time Complexities

Time Complexity	Description	Example
O(1)	Constant time	Accessing an array element
O(log n)	Logarithmic time	Binary Search
O(n)	Linear time	Loop through an array
O(n log n)	Log-linear time	Merge Sort, Quick Sort (average)
$O(n^2)$	Quadratic time	Nested loops
$O(2^n)$	Exponential time	Recursive Fibonacci
O(n!)	Factorial time	N-Queens (brute force)

What is Space Complexity?

Space Complexity measures how much **extra memory** an algorithm uses relative to input size **n**.

It includes:

- Function call stack
- Auxiliary arrays or data structures
- Variables used in computation

(Input storage is usually not counted.)

Common Space Complexities

Space Complexity	Description	Example
O(1)	Constant space	Few variables only
O(n)	Linear space	Extra array
O(log n)	Logarithmic space	Binary Search recursion
$O(n^2)$	Quadratic space	Matrix / 2D array problems

Example 1: Linear Search

```
def find(arr, x):
    for i in arr:
        if i == x:
            return True
    return False
```

Time Complexity: O(n) — One loop through array
Space Complexity: O(1) — Only few variables used

Example 2: Recursive Fibonacci

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

Time Complexity: $O(2^n)$ — Many repeated recursive calls
Space Complexity: O(n) — Recursion call stack depth

How to Calculate Time Complexity?

1. Count loops and recursive calls.
2. Focus on the dominant term.
3. Ignore constants.
   Example: O(2n) → O(n)

How to Calculate Space Complexity?

1. Count extra memory used.
2. Include arrays, recursion stack, hash maps, etc.
3. Ignore input storage.

Beginner Tips

- Single loop → **O(n)**
- Nested loops → **$O(n^2)$**
- Recursion → consider call stack size
- Efficient sorting → **O(n log n)**
- Hashing access → usually **O(1)**